DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Subject Name: **DATABASE MANAGEMENT SYSTEMS** Subject Code: **CS T53**

UNIT –III

Relational Database Design: Features of Good Relational Designs – Atomic Domains and First Normal Form – Second Normal Form – Decomposition Using Functional Dependencies – Functional Dependency Theory – Algorithms for decomposition – Decomposition Using Multi-valued Dependencies – More Normal Forms – Database – Design Process – Modeling Temporal Data

.

TWO MARK

1. **What is normalization? (NOV 2015)**

   The main goal of normalization is to reduce redundant data. Normalization is based on functional dependencies.

2. **What is First Normal Form?(NOV 2010)**

   First normal form is also called as flat file. There are no composite attributes and every attribute is single and describe one property.

3. **What is functional dependencies?(NOV 2015) [NOV 2017], [APR 2018]**

   Functional dependencies play key role in differentiating good database designs from bad database design. A functional dependency is a type of constraint that is a generalization of the notion of key.

4. **What is decomposition?**

   The process of decomposing a relation schema that has many attributes into several schemas with fewer attributes is called decomposition.

5. **What is Boyce Codd Normal Form? APRIL 2014**

   A relation schema R is in BCNF with respect to a set F of functional dependencies if, for all functional dependencies in F^+ of the form $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, atleast one of the following holds:

- $\alpha \rightarrow \beta$ is a trivial functional dependency.
- α is a superkey for schema R.

6. What is Third Normal Form?

A relation is said to be Third normal form where all attributes in a relation tuple are not only functionally dependent on key attributes but also dependent on non key attributes.

7. What is Second Normal Form?

A relation is said to be in Second normal form, if it is in first normal form and non key attributes are functionally dependent on the key attributes.

8. What is Fourth Normal Form?

This schema is called as non- BCNF schema. A relation defined with multi valued dependency which is a new form of constraint is called forth normal form. It is more restrictive than BCNF.

9. What is multi valued dependency?

Multivalued dependencies are referred to as tuple generating dependencies. It do not rule out existence of certain tuples, instead they require other tuples of certain form be present in the relation.

10. Comparison of BCNF and 3CNF.

3CNF	BCNF
• 3NF: For every functional dependency X->Y in a set *F* of functional dependencies over relation *R*, either: Y is a subset of X or,X is a *superkey* of R, or Y is a subset of K for some key K of R *N.b.,* no subset of a key is a key. • It is always possible to obtain a 3NF design without sacrificing lossless-join or dependency preservation. • If we do not eliminate all transitive dependencies, we may need to use null values to represent some of the meaningful relationships. • Repetition of information occurs.	• BCNF: For every functional dependency X->Y in a set *F* of functional dependencies over relation *R*, either: Y is a subset of X or, X is a *superkey* of R • If we cannot check for dependency preservation eciently, we either pay a high price in system performance or risk the integrity of the data. • The limited amount of redundancy in 3NF is then a lesser evil.

11. Define Functional Dependencies. (DEC2018)

Let $X \subseteq R$ and $Y \subseteq R$, then the Functional dependency (FD) $X \rightarrow Y$ holds on R if, in any relation r(R), for all pairs of tuples t1 and t2 in r such that t1[X] = t2[X], it is also the case that t1[Y] = t2[Y].

12. Define Closure of Functional Dependency.

Let F be a set of FDs, then the closure of F is the set of all FDs logically implied by F. It is denoted by F^+

13. Define normalization of data and denormalization.

Normalization:

Process of decomposing an unsatisfactory relation schema into satisfactory relation schemas by breaking their attributes, so as to satisfy the desirable properties. Denormalization: Process of storing the join of higher normal form relations as base relation, which is in the lower normal form.

14. Explain shortly the four properties or objectives of normalization. [NOV 2017], (DEC2018)

- To minimize redundancy
- To minimize insertion, deletion, and updating anomalies.
- Lossless-join or non-additive join decomposition
- Dependency preservation

15. Define Partial Functional Dependency.

A FD $x \rightarrow y$ is a partial dependency if some attribute $A \in x$ can be removed from x and the dependency still holds; ie., for some $A \in x$, $(x - \{ A \}) \rightarrow y$.

16. Define Boyce codd normal form

A relation schema R is in BCNF with respect to a set F of functional+ dependencies if, for all functional dependen cies in F of the form. a->ß, where a

17. List the disadvantages of relational database system

Repetition of data

Inability to represent certain information.

18. What is first normal form?

The domain of attribute must include only atomic (simple, indivisible) values.

19. What are the uses of functional dependencies?

To test relations to see whether they are legal under a given set of functional dependencies. To specify constraints on the set of legal relations.

20. Explain trivial dependency?

Functional dependency of the form a ß is trivial if ß C a. Trivial functional dependencies are satisfied by all the relations.

21. What are axioms?

Axioms or rules of inference provide a simpler technique for reasoning about functional dependencies.

22. What is meant by computing the closure of a set of functional dependency?

+ The closure of F denoted b y F is the set of functional dependencies logically implied by F.

23. What is meant by normalization of data?

It is a process of analyzing the given relation schemas based on their Functional Dependencies (FDs) and primary key to achieve the properties Minimizing redundancy Minimizing insertion, deletion and updating anomalies

24. Explain the desirable properties of decomposition.

a) Lossless-join decomposition b) Dependency preservation c) Repetition of information

25. What is 2NF?

A relation schema R is in 2NF if it is in 1NF and every non-prime attribute A in R is fully functionally dependent on primary key.

26. Define Redundancy?

To store extra information that is not needed normally, but that can be used in the event of failure of the disk to rebuild the lost information.

27. What is meant by Dependency preservation? (April 2013)

Another desirable property in database design is **dependency preservation**.

- We would like to check easily that updates to the database do not result in illegal relations being created.
- It would be nice if our design allowed us to check updates without having to compute natural joins.
- To know whether joins must be computed, we need to determine what functional dependencies may be tested by checking each relation individually.
- Let *F* be a set of functional dependencies on schema *R*.
- Let $\{R_1, R_2, \ldots, R_n\}$ be a decomposition of *R*.

28. State the conditions for a relation R to be in 3NF (April 2013)

A relation R id in 3NF if and only if for every non trivial FD A ->B for R, one of the following two conditions are true:

- A is a super key for R

- B is an attribute in some keys

29. When we say a relation is in first Normal form? (Nov 2010)

1NF A relation R is in first normal form (1NF) if and only if all underlying domains contain atomic values only.

Example: 1NF but not 2NF

FIRST (supplier_no, status, city, part_no, quantity)

Functional Dependencies:

(supplier_no, part_no) □ quantity

(supplier_no) □ status

(supplier_no) □ city

city □ status (Supplier's status is determined by location)

30. Explain the rules in Armstrong axioms [APR 2018]

Armstrong's axioms. **Armstrong's axioms** are a set of **axioms** (or, more precisely, inference **rules**) used to infer all the functional dependencies on a relational database. he axioms are sound in generating only functional dependencies in the closure of a set of functional dependencies (denoted as F+ when applied to that set (denoted as F+). They are also complete in that repeated application of these rules will generate all functional dependencies in the closure F+

11 MARKS

1. Explain in detail about Relational database design? (11 Marks)

First Normal Form

A domain is **atomic** if elements of the domain are considered to be indivisible units. We say that a relation schema R is in **first normal form** (1NF) if the domains of all attributes of R are atomic. A set of names is an example of a nonatomic value. For example, if the schema of a relation employee included an attribute childrenwhose domain elements are sets of names, the schema would not be in first normal form. Composite attributes, such as an attribute address with component attributes street and city, also have nonatomic domains. Integers are assumed to be atomic, so the set of integers is an atomic domain; the set of all sets of integers is a nonatomic domain. The distinction is that we do not normally consider integers to have subparts, but we consider sets of integers to have subparts namely, the integers making up the set.

The domain of all integers would be nonatomic if we considered each integer to be an ordered list of digits.

Some types of nonatomic values can be useful, although they should be used with care. For example, composite valued attributes are often useful, and set valued attributes are also useful in many cases, which is why both are supported in the E-R model.

Pitfalls in Relational-Database Design

Among the undesirable properties that a bad design may have are:

- Repetition of information
- Inability to represent certain information

Suppose the information concerning loans is kept in one single relation, lending, which is defined over the relation schema

Lending-schema = (branch-name, branch-city, assets, customer-name,
loan-number, amount)

A tuple t in the lending relation has the following intuitive meaning:

- t[assets] is the asset figure for the branch named t[branch-name].
- t[branch-city] is the city in which the branch named t[branch-name] is located.
- t[loan-number] is the number assigned to a loan made by the branch named t[branch-name] to the customer named t[customer-name].

- t[amount] is the amount of the loan whose number is t[loan-number].

Suppose that we wish to add a new loan to our database. Say that the loan is made by the Perryridge branch to Adams in the amount of $1500. Let the loan-number be L-31. In our design, we need a tuple with values on all the attributes of Lendingschema. Thus, we must repeat the asset and city data for the Perryridge branch, and must add the tuple

(Perryridge, Horseneck, 1700000, Adams, L-31, 1500)

Another problem with the Lending-schema design is that we cannot represent directly the information concerning a branch (branch-name, branch-city, assets) unless there exists at least one loan at the branch. This is because tuples in the lending relation require values for loan-number, amount, and customer-name. One solution to this problem is to introduce null values, as we did to handle updates through views.

branch-name	*branch-city*	*assets*	*customer-name*	*loan-number*	*amount*
Downtown	Brooklyn	9000000	Jones	L-17	1000
Redwood	Palo Alto	2100000	Smith	L-23	2000
Perryridge	Horseneck	1700000	Hayes	L-15	1500
Downtown	Brooklyn	9000000	Jackson	L-14	1500
Mianus	Horseneck	400000	Jones	L-93	500
Round Hill	Horseneck	8000000	Turner	L-11	900
Pownal	Bennington	300000	Williams	L-29	1200
North Town	Rye	3700000	Hayes	L-16	1300
Downtown	Brooklyn	9000000	Johnson	L-18	2000
Perryridge	Horseneck	1700000	Glenn	L-25	2500
Brighton	Brooklyn	7100000	Brooks	L-10	2200

Sample lending relation.

Functional Dependencies

Basic Concepts

Functional dependencies are constraints on the set of legal relations. They allow us to express facts about the enterprise that we are modeling with our database.

Let R be a relation schema. A subset K of R is a **superkey**of R if, in any legal relation r(R), for all pairs t1 and t2 of tuples in r such that t1 _= t2, then t1[K] _= t2[K]. That is, no two tuples in any legal relation r(R) may have the same value on attribute set K. The notion of functional dependency generalizes the notion of superkey. Consider a relation schema R, and let α ⊆R and β ⊆R. The **functional dependency**

α→β

holds on schema R if, in any legal relation r(R), for all pairs of tuples t1 and t2 in r such that t1[α] = t2[α], it is also the case that t1[β] = t2[β].

Using the functional-dependency notation, we say that K is a superkey of R if K → R. That is, K is a superkey if, whenever t1[K] = t2[K], it is also the case that t1[R] = t2[R] (that is, t1 = t2).Functional dependencies allow us to express constraints that we cannot express with superkeys. Consider the schema

Loan-info-schema = (loan-number, branch-name, customer-name, amount)

which is simplification of the Lending-schema that we saw earlier. The set of functional dependencies that we expect to hold on this relation schema is

loan-number→amount

loan-number→branch-name

We shall use functional dependencies in two ways:

1. To test relations to see whether they are legal under a given set of functional dependencies. If a relation r is legal under a set F of functional dependencies, we say that r **satisfies** F.
2. To specify constraints on the set of legal relations. We shall thus concern ourselves with only those relations that satisfy a given set of functional dependencies. If we wish to constrain ourselves to relations on schema R that satisfy a set F of functional dependencies, we say that F **holds** on R.

To distinguish between the concepts of a relation satisfying a dependency and a dependency holding on a schema, we return to the banking example. In the loan relation (on Loan-schema) we see that the dependency loannumber→ amount is satisfied. In contrast to the case of customer-city and customerstreetin Customer-schema.

customer-name	customer-street	customer-city
Jones	Main	Harrison
Smith	North	Rye
Hayes	Main	Harrison
Curry	North	Rye
Lindsay	Park	Pittsfield
Turner	Putnam	Stamford
Williams	Nassau	Princeton
Adams	Spring	Pittsfield
Johnson	Alma	Palo Alto
Glenn	Sand Hill	Woodside
Brooks	Senator	Brooklyn
Green	Walnut	Stamford

The customer relation.

loan-number	branch-name	amount
L-17	Downtown	1000
L-23	Redwood	2000
L-15	Perryridge	1500
L-14	Downtown	1500
L-93	Mianus	500
L-11	Round Hill	900
L-29	Pownal	1200
L-16	North Town	1300
L-18	Downtown	2000
L-25	Perryridge	2500
L-10	Brighton	2200

The loan relation.

In the branch relation of Figure 7.5, we see that branch-name → assets is satisfied, as is assets → branch-name. We want to require that branch-name → assets hold on Branch-schema. However, we do not wish to require that assets → branch-name hold, since it is possible to have several branches that have the same asset value.

branch-name	branch-city	assets
Downtown	Brooklyn	9000000
Redwood	Palo Alto	2100000
Perryridge	Horseneck	1700000
Mianus	Horseneck	400000
Round Hill	Horseneck	8000000
Pownal	Bennington	300000
North Town	Rye	3700000
Brighton	Brooklyn	7100000

The branch relation.

- On Branch-schema: branch-name →branch-city
  branch-name→assets

- On Customer-schema:customer-name→ customer-city
  customer-name→ customer-street

- On Loan-schema: loan-number → amount
  loan-number→ branch-name

- On Borrower-schema: No functional dependencies
- On Account-schema: account-number → branch-name
  account-number→ balance
- On Depositor-schema: No functional dependencies

Closure of a Set of Functional Dependencies

More formally, given a relational schema R, a functional dependency f on R is **logically implied** by a set of functional dependencies F on R if every relation instancer(R) that satisfies F also satisfies f.
Suppose we are given a relation schema R = (A, B, C, G, H, I) and the set of functional dependencies
A →B
A →C
CG→ H
CG→ I
B → H
The functional dependency
A→ H

is logically implied. That is, we can show that, whenever our given set of functional dependencies holds on a relation, A→H must also hold on the relation. Suppose that t1 and t2 are tuples such that
t1[A] = t2[A]
Since we are given that A→B, it follows from the definition of functional dependency that
t1[B] = t2[B]
Then, since we are given that B → H, it follows from the definition of functional dependency that
t1[H] = t2[H]
Therefore, we have shown that, whenever t1 and t2 are tuples such that t1[A] = t2[A], it must be that t1[H] = t2[H]. But that is exactly the definition of A→ H.

We can use the following three rules to find logically implied functional dependencies. By applying these rules repeatedly, we can find all of F+, given F. This collection of rules is called **Armstrong's axioms** in honor of the person who first proposed it.

- **Reflexivity rule**. If α is a set of attributes and $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$ holds.
- **Augmentation rule**. If $\alpha \rightarrow \beta$ holds and γ is a set of attributes, then $\gamma\alpha \rightarrow \gamma\beta$ holds.
- **Transitivity rule**. If $\alpha \rightarrow \beta$ holds and $\beta \rightarrow \gamma$ holds, then $\alpha \rightarrow \gamma$ holds.

Although Armstrong's axioms are complete, it is tiresome to use them directly for the computation of F+. To simplify matters further, we list additional rules. It is possible to use Armstrong's axioms to prove that these rules are correct .

- **Union rule**. If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds.
- **Decomposition rule**. If $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds.
- **Pseudotransitivity rule**. If $\alpha \rightarrow \beta$ holds and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds.

Closure of Attribute Sets

To test whether a set α is a superkey, we must devise an algorithm for computing the set of attributes functionally determined by α. One way of doing this is to compute F+, take all functional dependencies with α as the left-hand side, and take the union of the right-hand sides of all such dependencies.However, doing so can be expensive, since F+ can be large.

F+ = F
repeat
for each functional dependency f in F+
apply reflexivity and augmentation rules on f
add the resulting functional dependencies to F+
for each pair of functional dependencies f1 and f2 in F+
iff1 and f2 can be combined using transitivity
add the resulting functional dependency to F+
untilF+ does not change any further

Let α be a set of attributes.We call the set of all attributes functionally determined by α under a set F of functional dependencies the **closure** of α under F; we denote it by α+. The input is a set F of functional

dependencies and the set α of attributes. The output is stored in the variable result. The first time that we execute the **while** loop to test each functional dependency, we find that

• A → B causes us to include B in result. To see this fact, we observe that A → B is in F, A ⊆result (which is AG), so result :=result ∪B.

• A→ C causes result to become ABCG.

• CG→H causes result to become ABCGH.

• CG→I causes result to become ABCGHI.

It turns out that, in the worst case, this algorithm may take an amount of time quadratic in the size of F. There is a faster (although slightly more complex) algorithm that runs in time linear in the size of F.

result:= α;
while(changes to result) **do**
for each functional dependency β → γ **in** F **do**
begin
ifβ ⊆result **then** result := result ∪γ;
end

There are several uses of the attribute closure algorithm:

- To test if α is a superkey, we compute α+, and check if α+ contains all attributes of R.
- We can check if a functional dependency α → β holds (or, in other words, is in F+), by checking if β ⊆α+. That is, we compute α+ by using attribute closure, and then check if it contains β.
- It gives us an alternative way to compute F+: For each γ ⊆R, we find the closure γ+, and for each S ⊆γ+, we output a functional dependency γ → S.

Canonical Cover

Suppose that we have a set of functional dependencies F on a relation schema. Whenever a user performs an update on the relation, the database system must ensure that the update does not violate any functional dependencies, that is, all the functional dependencies in F are satisfied in the new database state. The system must roll back the update if it violates any functional dependencies in the set F.

An attribute of a functional dependency is said to be **extraneous** if we can remove it without changing the closure of the set of functional dependencies. The formal definition of **extraneous attributes** is as follows. Consider a set F of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in F.

• Attribute A is extraneous in α if$A \in \alpha$, and F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.

• Attribute A is extraneous in β if $A \in \beta$, and the set of functional dependencies $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies F.

Consider an attribute A in a dependency $\alpha \rightarrow \beta$.

• If $A \in \beta$, to check if A is extraneous consider the set $F_ = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$and check if $\alpha \rightarrow A$ can be inferred fromF_. To do so, compute $\alpha+$ (the closure of α) under F_; if $\alpha+$ includes A, then A is extraneous in β.

• If $A \in \alpha$, to check if A is extraneous, let $\gamma = \alpha - \{A\}$, and check if $\gamma \rightarrow \beta$can be inferred from F. To do so, compute $\gamma+$ (the closure of γ) under F; if $\gamma+$includes all attributes in β, then A is extraneous in α.

A **canonical cover** Fc for F is a set of dependencies such that F logically implies all dependencies in Fc, and Fc logically implies all dependencies in F. Furthermore, Fcmust have the following properties:

• No functional dependency in Fc contains an extraneous attribute.

• Each left side of a functional dependency in Fc is unique. That is, there are no two dependencies $\alpha 1 \rightarrow \beta 1$ and $\alpha 2 \rightarrow \beta 2$ in Fc such that $\alpha 1 = \alpha 2$.

2 .Explain Decomposition? (11 Marks) [NOV 2017] (DEC2018)(MAY 2019)

Consider an alternative design in which we decompose Lending-schema into the following two schemas:

Branch-customer-schema = (branch-name, branch-city, assets, customer-name)

Customer-loan-schema = (customer-name, loan-number, amount)

Using the lending relation of Figure 7.1,we construct our new relations branch-customer (Branch-customer) and customer-loan (Customer-loan-schema):

branch-customer= Пbranch-name, branch-city, assets, customer-name (lending)

customer-loan= Пcustomer-name, loan-number, amount (lending)

It appears that we can do so by writing

branch-customer□ customer-loan

branch-name	*branch-city*	*assets*	*customer-name*
Downtown	Brooklyn	9000000	Jones
Redwood	Palo Alto	2100000	Smith
Perryridge	Horseneck	1700000	Hayes
Downtown	Brooklyn	9000000	Jackson
Mianus	Horseneck	400000	Jones
Round Hill	Horseneck	8000000	Turner
Pownal	Bennington	300000	Williams
North Town	Rye	3700000	Hayes
Downtown	Brooklyn	9000000	Johnson
Perryridge	Horseneck	1700000	Glenn
Brighton	Brooklyn	7100000	Brooks

The relation branch-customer.

customer-name	*loan-number*	*amount*
Jones	L-17	1000
Smith	L-23	2000
Hayes	L-15	1500
Jackson	L-14	1500
Jones	L-93	500
Turner	L-11	900
Williams	L-29	1200
Hayes	L-16	1300
Johnson	L-18	2000
Glenn	L-25	2500
Brooks	L-10	2200

The relation customer-loan.

branch-name	*branch-city*	*assets*	*customer-name*	*loan-number*	*amount*
Downtown	Brooklyn	9000000	Jones	L-17	1000
Downtown	Brooklyn	9000000	Jones	L-93	500
Redwood	Palo Alto	2100000	Smith	L-23	2000
Perryridge	Horseneck	1700000	Hayes	L-15	1500
Perryridge	Horseneck	1700000	Hayes	L-16	1300
Downtown	Brooklyn	9000000	Jackson	L-14	1500
Mianus	Horseneck	400000	Jones	L-17	1000
Mianus	Horseneck	400000	Jones	L-93	500
Round Hill	Horseneck	8000000	Turner	L-11	900
Pownal	Bennington	300000	Williams	L-29	1200
North Town	Rye	3700000	Hayes	L-15	1500
North Town	Rye	3700000	Hayes	L-16	1300
Downtown	Brooklyn	9000000	Johnson	L-18	2000
Perryridge	Horseneck	1700000	Glenn	L-25	2500
Brighton	Brooklyn	7100000	Brooks	L-10	2200

The relation branch-customer □ customer-loan.

Desirable Properties of Decomposition

We illustrate our concepts with the Lendingschema

Lending-schema = (branch-name, branch-city, assets, customer-name,

loan-number, amount)

The set F of functional dependencies that we require to hold on Lending-schema are

branch-name→ branch-city assets

loan-number→ amount branch-name

Lending-schema is an example of a bad database design. Assume that we decompose it to the following three relations:

Branch-schema = (branch-name, branch-city, assets)

Loan-schema = (loan-number, branch-name, amount)

Borrower-schema = (customer-name, loan-number)

Lossless-Join Decomposition

Let R be a relation schema, and let F be a set of functional dependencies on R. Let R1 and R2 form a decomposition of R. This decomposition is a lossless-join decomposition of R if at least one of the following functional dependencies is in F+:

• R1 ∩ R2 → R1

• R1 ∩ R2 → R2

In other words, if R1 ∩ R2 forms a superkey of either R1 or R2, the decomposition of R is a lossless-join decomposition.

We now demonstrate that our decomposition of Lending-schema is a lossless-join decomposition by showing a sequence of steps that generate the decomposition. We begin by decomposing Lending-schema into two schemas:

Branch-schema = (branch-name, branch-city, assets)

Loan-info-schema = (branch-name, customer-name, loan-number, amount)

Since branch-name → branch-city assets, the augmentation rule for functional dependencies implies that

branch-name→ branch-name branch-city assets

Since Branch-schema ∩ Loan-info-schema = {branch-name}, it follows that our initial decomposition is a lossless-join decomposition. Next, we decompose Loan-info-schema into

Loan-schema = (loan-number, branch-name, amount)

Borrower-schema = (customer-name, loan-number)

This step results in a lossless-join decomposition, since loan-number is a common attribute and **loan-number → amount branch-name.**

Dependency Preservation

To decide whether joins must be computed to check an update, we need to determine what functional dependencies can be tested by checking each relation individually. Let F be a set of functional dependencies on a schema R, and let R1, R2, . . . ,Rnbe a decomposition of R. The **restriction** of F to Riis the set Fi of all functional dependencies in F+ that include only attributes of Ri. Since all functional dependencies in a restriction involve attributes of only one relation schema, it is possible to test such a dependency for satisfaction by checking only one relation.

We consider each member of the set F of functional dependencies that we require to hold on Lending-schema, and show that each one can be tested in at least one relation in the decomposition.

• We can test the functional dependency: branch-name → branch-city assets using Branch-schema = (branch-name, branch-city, assets)

• We can test the functional dependency: loan-number → amount branch-name using Loan-schema = (branch-name, loan-number, amount).

computeF+;
for each schema Riin D **do**
begin
Fi : = the restriction of F+ to Ri;
end
F_ := ∅
for each restriction Fi **do**
begin
F_ = F_ ∪Fi
end
computeF_+;
if(F_+ = F+) **then** return (true)
elsereturn (false);

Repetition of Information

In the decomposition, the relation on schema Borrowerschemacontains the loan-number, customer-name relationship, and no other schemadoes. Therefore, we have one tuple for each customer for a loan in only the relationon Borrower-schema. In the other relations involving loan-number (those on schemasLoan-schema and Borrower-schema), only one tuple per loan needs to appear.

3. Explain in detail about Boyce–Codd Normal Form? (11 Marks)

Definition

One of the more desirable normal forms that we can obtain is **Boyce–Codd normal form** (**BCNF**). A relation schema R is in BCNF with respect to a set F of functionaldependencies if, for all functional dependencies in F+ of the form $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds:

- $\alpha \rightarrow \beta$ is a trivial functional dependency (that is, $\beta \subseteq \alpha$).
- $\alpha$is a superkey for schema R.

A database design is in BCNF if each member of the set of relation schemas that constitutes the design is in BCNF. Consider the following relation schemas and their respective functional dependencies:

- Customer-schema = (customer-name, customer-street, customer-city) customer-name → customer-street customer-city
- Branch-schema = (branch-name, assets, branch-city) branch-name → assets branch-city
- Loan-info-schema = (branch-name, customer-name, loan-number, amount) loan-number → amount branch-name

It is now possible to avoid redundancy in the case where there are several customers associated with a loan. There is exactly one tuple for each loan in the relation on Loan-schema, and one tuple for each customer of each loan in the relation on Borrower-schema. Thus, we do not have to repeat the branch name and the amount once for each customer associated with a loan. Often testing of a relation to see if it satisfies BCNF can be simplified:

- To check if a nontrivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF, computeα+ (the attribute closure of α), and verify that it includes all attributes ofR; that is, it is a superkey of R.
- To check if a relation schema R is in BCNF, it suffices to check only the dependencies in the given set F for violation of BCNF, rather than check all dependenciesin F

Decomposition Algorithm

The decomposition that the algorithm generates is not only in BCNF, but is also a lossless-join decomposition. To see why our algorithm generates only lossless-join decompositions, we note that, when we replace a schema Riwith (Ri−β) and (α, β), the dependency α →β holds, and (Ri−β) ∩(α, β) = α.

result:= {R};

done:= false;

computeF+;

while(**not** done) **do**

if(there is a schema Riin result that is not in BCNF)

then begin

letα →β be a nontrivial functional dependency that holds

onRisuch that α →Riis not in F+, and α ∩β = ∅;

result:= (result −Ri) ∪(Ri−β) ∪(α, β);

end

elsedone := true;

Lending-schema = (branch-name, branch-city, assets, customer-name,loan-number, amount)

The set of functional dependencies that we require to hold on Lending-schema are

branch-name→assets branch-city

loan-number→amount branch-name

A candidate key for this schema is {loan-number, customer-name}.

We can apply the algorithm of Figure 7.13 to the Lending-schema example as follows:

•The functional dependency

branch-name→assets branch-city

holds on Lending-schema, but branch-name is not a superkey. Thus, Lendingschemais not in BCNF. We replace Lending-schema by

Branch-schema = (branch-name, branch-city, assets)

Loan-info-schema = (branch-name, customer-name, loan-number, amount)

Dependency Preservation

Banker-schema = (branch-name, customer-name, banker-name)

which indicates that a customer has a "personal banker" in a particular branch. The set F of functional dependencies that we require to hold on the Banker-schema is

banker-name → branch-name

branch-name customer-name → banker-name

Clearly, Banker-schema is not in BCNF since banker-name is not a superkey. If we apply the algorithm we obtain the following BCNF decomposition:

Banker-branch-schema = (banker-name, branch-name)

Customer-banker-schema = (customer-name, banker-name)

The decomposed schemas preserve only banker-name → branch-name (and trivial dependencies), but the closure of {banker-name → branch-name} does not include customer-name branch-name → banker-name. The violation of this dependency cannot be detected unless a join is computed.

Thus, the example shows that we cannot always satisfy all three design goals:

1. Lossless join

2. BCNF

3. Dependency preservation

4 .Explain about third normal form? (11 Marks)APR 2014

We have two alternatives if we wish to check if an update violates any functional dependencies:

• Pay the extra cost of computing joins to test for violations.

• Use an alternative decomposition, third normal form (3NF), which we present below, which makes testing of updates cheaper. Unlike BCNF, 3NF decompositions may contain some redundancy in the decomposed schema.

Definition

BCNF requires that all nontrivial dependencies be of the form $\alpha \rightarrow \beta$, where α is a superkey. 3NF relaxes this constraint slightly by allowing nontrivial functional dependencies whose left side is not a superkey.

A relation schema R is in **third normal form (3NF)** with respect to a set F of functional dependenciesif, for all functional dependencies in F+ of the form $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds:

- $\alpha \rightarrow \beta$ is a trivial functional dependency.
- α is a superkey for R.
- Each attribute A in $\beta - \alpha$ is contained in a candidate key for R.

The first two alternatives are the same as the two alternatives in the definition of BCNF. The third alternative of the 3NF definition seems rather unintuitive, and it is not obvious why it is useful. It represents, in some sense, a minimal relaxation of the BCNF conditions that helps ensure that every schema has a dependency-preserving decomposition into 3NF. Its purpose will become more clear later, when we study decomposition into 3NF.

The only nontrivial functional dependencies of the form $\alpha \rightarrow$ banker-name include {customer-name, branch-name} as part of α. Since {customer-name, branch-name} is a candidate key, these dependencies do not violate the definition of 3NF.

Decomposition Algorithm

The set of dependencies Fc used in the algorithm is a canonical cover for F.Note that the algorithm considers the set of schemas Rj, j = 1, 2, . . . , i; initially i = 0, and in this case the set is empty.

let Fc be a canonical cover for F;
i := 0;
for each functional dependency $\alpha \rightarrow \beta$ in Fc **do**
if none of the schemas Rj, j = 1, 2, . . . , i contains $\alpha\beta$
then begin
i := i + 1;
Ri := $\alpha\beta$;
end
if none of the schemas Rj, j = 1, 2, . . . , i contains a candidate key for R
then begin
i := i + 1;
Ri := any candidate key for R;
end
return (R1,R2, . . .,Ri)
Banker-info-schema = (branch-name, customer-name, banker-name, office-number)
The main difference here is that we include the banker's office number as part of the information. The functional dependencies for this relation schema are
banker-name $\rightarrow$ branch-name office-number

customer-name branch-name → banker-name

The **for** loop in the algorithm causes us to include the following schemas in our decomposition:

Banker-office-schema = (banker-name, branch-name, office-number)

Banker-schema = (customer-name, branch-name, banker-name)

Let us consider the three possible cases:

- B is in both α and β. In this case, the dependency α → β would not have been in Fc since B would be extraneous in β. Thus, this case cannot hold.
- B is in β but not α. Consider two cases:

  - γ is a superkey. The second condition of 3NF is satisfied.

  - γ is not a superkey. Then α must contain some attribute not in γ. Now, since γ → B is in F+, it must be derivable from Fc by using the attribute closure algorithm on γ. The derivation could not have used α → β— if it had been used, α must be contained in the attribute closure of γ, which is not possible, since we assumed γ is not a superkey. Now, using α → (β −{B}) and γ → B, we can derive α → B (since γ ⊆ αβ, and γ cannot contain B because γ → B is nontrivial). This would imply that B is extraneous in the right hand side of α → β, which is not possible since α → β is in the canonical cover Fc. Thus, if B is in β, then γ must be a superkey, and the second condition of 3NF must be satisfied.
- B is in α but not β. Since α is a candidate key, the third alternative in the definition of 3NF is satisfied.

5. Explain about fourth normal form? (11 Marks)APR 2014

BC-schema = (loan-number, customer-name, customer-street, customer-city)

The astute reader will recognize this schema as a non-BCNF schema because of the functional dependency

customer-name → customer-street customer-city

Multivalued Dependencies

Functional dependencies rule out certain tuples from being in a relation. If A → B, then we cannot have two tuples with the same A value but different B values. Multivalued dependencies, on the other hand, do not rule out the existence of certain tuples. Instead, they require that other tuples of a certain form be present in the relation. For this reason, functional dependencies sometimes are referred to as **equalitygenerating dependencies**, and multivalued dependencies are referred to as **tuplegenerating dependencies**.

Let R be a relation schema and let α ⊆ R and β ⊆ R. The **multivalued dependency**

α →→ β

holds on R if, in any legal relation r(R), for all pairs of tuples t1 and t2 in r such that t1[α] = t2[α], there exist tuples t3 and t4 in r such that

t1[α] = t2[α] = t3[α] = t4[α]
t3[β] = t1[β]
t3[R − β] = t2[R − β]
t4[β] = t2[β]
t4[R − β] = t1[R − β]

As with functional dependencies, we shall use multivalued dependencies in two ways:
1. To test relations to determine whether they are legal under a given set of functional and multivalued dependencies
2. To specify constraints on the set of legal relations; we shall thus concern ourselves with only those relations that satisfy a given set of functional and multivalued dependencies.

From the definition of multivalued dependency, we can derive the following rule:
• If α → β, then α →→ β.
In other words, every functional dependency is also a multivalued dependency.

Definition of Fourth Normal Form

Consider again our BC-schema example in which the multivalued dependency customer name→→customer-street customer-city holds, but no nontrivial functional dependencies hold.We saw in the opening paragraphs of Section 7.8 that, although BCschema is in BCNF, the design is not ideal, since we must repeat a customer's address information for each loan. We shall see that we can use the given multivalued dependency to improve the database design, by decomposing BC-schema into a **fourth normal form** decomposition.

A relation schema R is in **fourth normal form** (4NF) with respect to a set D of functional and multivalued dependencies if, for all multivalued dependencies in D+ of the form α →→ β, where α ⊆ R and β ⊆ R, at least one of the following holds
• α →→ β is a trivial multivalued dependency.
• α is a superkey for schema R.
A database design is in 4NF if each member of the set of relation schemas that constitutes the design is in 4NF.

result := {R};

done := false;

compute D+; Given schema Ri, let Di denote the restriction of D+ to Ri

while (**not** done) **do**

if (there is a schema Ri in result that is not in 4NF w.r.t. Di)

then begin

let $\alpha \rightarrow\rightarrow \beta$ be a nontrivial multivalued dependency that holds

on Ri such that $\alpha \rightarrow Ri$ is not in Di, and $\alpha \cap \beta = \emptyset$;

result := (result − Ri) ∪ (Ri − β) ∪ (α, β);

end

else done := true;

Let R be a relation schema, and let R1,R2, . . . , Rn be a decomposition of R. To check if each relation schema Ri in the decomposition is in 4NF, we need to find what multivalued dependencies hold on each Ri. Recall that, for a set F of functional dependencies, the restriction Fi of F to Ri is all functional dependencies in F+ that include only attributes ofRi.Now consider a setDof both functional and multivalued dependencies. The **restriction** of D to Ri is the set Di consisting of

1. All functional dependencies in D+ that include only attributes of Ri

2. All multivalued dependencies of the form

$\alpha \rightarrow\rightarrow \beta \cap Ri$

where $\alpha \subseteq Ri$ and $\alpha \rightarrow\rightarrow \beta$ is in D+.

Decomposition Algorithm

The analogy between 4NF and BCNF applies to the algorithm for decomposing a schema . It is identical to the BCNF decomposition algorithm of Figure 7.13, except that it uses multivalued, instead of functional, dependencies and uses the restriction of D+ to Ri. If we apply the algorithm BC-schema, we find that customer-name →→ loan-number is a nontrivial multivalued dependency, and customer-name is not a superkey for BC-schema. Following the algorithm, we replace BC-schema by two schemas:

Borrower-schema = (customer-name, loan-number)

Customer-schema = (customer-name, customer-street, customer-city).

This pair of schemas, which is in 4NF, eliminates the problem we encountered earlier with the redundancy of BC-schema.

Let R be a relation schema, and let D be a set of functional and multivalued dependencies on R. Let R1 and R2 form a decomposition of R. This decomposition is a lossless-join decomposition of R if and only if at least one of the following multivalued dependencies is in D+:

$R1 \cap R2 \rightarrow\rightarrow R1$

$R1 \cap R2 \rightarrow\rightarrow R2$

More Normal Forms

The fourth normal form is by no means the "ultimate" normal form. As we saw earlier, multivalued dependencies help us understand and tackle some forms of repetition of information that cannot be understood in terms of functional dependencies. There are types of constraints called **join dependencies** that generalize multivalued dependencies, and lead to another normal form called **project-join normal form (PJNF)** (PJNF is called **fifth normal form** in some books). There is a class of even more general constraints, which leads to a normal form called **domain-key normal form**.

6. Explain Overall Database Design Process? (11 Marks) NOV2014 [MAY 2019]

Several ways are:

1. R could have been generated when converting a E-R diagram to a set of tables.

2. R could have been a single relation containing all attributes that are of interest. The normalization process then breaks up R into smaller relations.

3. R could have been the result of some ad hoc design of relations, which we then test to verify that it satisfies a desired normal form.

E-R Model and Normalization

Suppose an employee entity had attributes department-number and department-address, and there is a functional dependency department-number → department-address. We would then need to normalize the relation generated from employee.

Functional dependencies can help us detect poor E-R design. If the generated relations are not in desired normal form, the problem can be fixed in the E-R diagram. That is, normalization can be done formally as part of data modeling. Alternatively, normalization can be left to the designer's intuition during E-R modeling, and can be done formally on the relations generated from the E-R model.

The Universal Relation Approach

The second approach to database design is to start with a single relation schema containing all attributes of interest, and decompose it. One of our goals in choosing a decomposition was that it be a lossless-join decomposition.

Dangling tuples may occur in practical database applications. They represent incomplete information, as they do in our example, wherewe wish to store data about a loan that is still in the process of being negotiated. The relation r1 □ r2 □ · · · □ rn is called a **universal relation**, since it involves all the attributes in the universe defined by R1 ∪ R2 ∪ · · · ∪ Rn.

The normal forms that we have defined generate good database designs from the point of view of representation of incomplete information. Returning again to the we would not want to allow storage of the following fact: "There is a loan (whose number is unknown) to Jones in the amount of $100." This is because

loan-number → customer-name amount and therefore the only way that we can relate customer-name and amount is through loan-number.

Denormalization for Performance

One alternative to computing the join on the fly is to store a relation containing all the attributes of account and depositor. Thismakes displaying the account information faster. However, the balance information for an account is repeated for every person who owns the account, and all copies must be updated by the application, whenever the account balance is updated. The process of taking a normalized schema and making it non-normalized is called **denormalization**, and designers use it to tune performance of systems to support time-critical operations.

Other Design Issues

Consider a company database, where we want to store earnings of companies in different years. A relation earnings(company-id, year, amount) could be used to store the earnings information. The only functional dependency on this relation is company-id, year→ amount, and the relation is in BCNF. An alternative design is to use multiple relations, each storing the earnings for a different year. Let us say the years of interest are 2000, 2001, and 2002; we would then have relations of the form earnings-2000, earnings-2001, earnings-2002, all of which are on the schema (company-id, earnings). The only functional dependency here on each relation would be company-id →earnings, so these relations are also in BCNF. However, this alternative design is clearly a bad idea—we would have to create a new relation every year, and would also have to write new queries every year, to take each new relation into account. Queries would also be more complicated since they may have to refer to many relations.

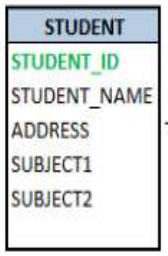
First Normal Form (1NF) (DEC2018)

A table is said to be in First Normal Form (1NF) if and only if each attribute of the relation is atomic.

That is,

- Each row in a table should be identified by primary key (a unique column value or group of unique column values)
- No rows of data should have repeating group of column values.

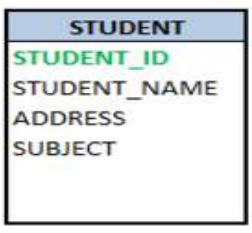
Let's consider the STUDENT table with his ID, Name address and 2 subjects that he has opted for.



STUDENT_ID	STUDENT_NAME	ADDRESS	SUBJECT1	SUBJECT 2
100	Joseph	Alaiedon Township	Mathematics	Physics
101	Allen	Fraser Township	Chemistry	Physics
102	Chris	Clinton Township	Mathematics	NULL
103	Patty	Troy	Physics	Mathematics

- Look at Chris entry. He has only subject. Hence Subject2 for him is NULL. Here storage space for second entry is simply wasted.
- In the case of Joseph, he has two subjects, Mathematics and Physics, both the columns have values. Imagine if he opts for third subject? There is no column for his third entry. In this case, whole table needs to be altered, which is not good at this stage. Once database is designed, it should be a perfect one. We should not be modifying it as we start adding/updating data.
- One of the requirements of 1NF is, each table should have primary key. This key in the table makes each record unique. In our example we have it already- STUDENT_ID.
- Here, SUBJECT1 and SUBJECT2 are same set of columns, i.e.; it has same kind of information stored - Subject, which is a violation of first rule of 1NF. As it states, there should not be any repeating columns. We have to remove such columns. But think how?

In order to have STUDENT in 1NF, we have to remove multiple SUBJECT columns from STUDENT table. Instead, create only one SUBJECT column, and for each STUDENT enters as many rows as SUBJECT he has. After making this change, the above table will change as follows:



	1NF		
STUDENT_ID	STUDENT_NAME	ADDRESS	SUBJECT
100	Joseph	Alaiedon Township	Mathematics
100	Joseph	Alaiedon Township	Physics
101	Allen	Fraser Township	Chemistry
101	Allen	Fraser Township	Physics
102	Chris	Clinton Township	Mathematics
103	Patty	Troy	Physics

Now STUDENT_ID alone cannot be a primary key, because it does not uniquely determines each record in the table. If we want to records for Joseph, and we query by his ID,100, gives us two records. Hence Student_ID is no more a primary key. When we observe the data in the table, all the four field uniquely determines each record. Hence all four fields together considered as primary key.

Thus, the above table is in 1NF form.

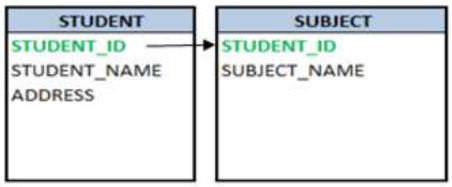
Second Normal Form (2NF)

A relation is said to be in a second normal form if and only if,

- it's in first normal form
- Every non-key attributes are identified by the use of primary key
- All subset of data, which applies to have multiple rows in a table must be removed and placed in a new table. And this new table and the parent table should be related by the use of foreign key.

In the 1NF STUDENT table above, Joseph and Allen have multiple rows because of their SUBJECTS. Although it is in 1NF form, it wastes storage space by repeating whole of their information - name and address in each row. In addition, Student ID alone is strong enough to be a primary key. If we make Student_ID as primary, all other attributes in the table cannot be uniquely identified. This is because of multiple rows exists for single ID. Hence it does not satisfy second condition of 2NF.

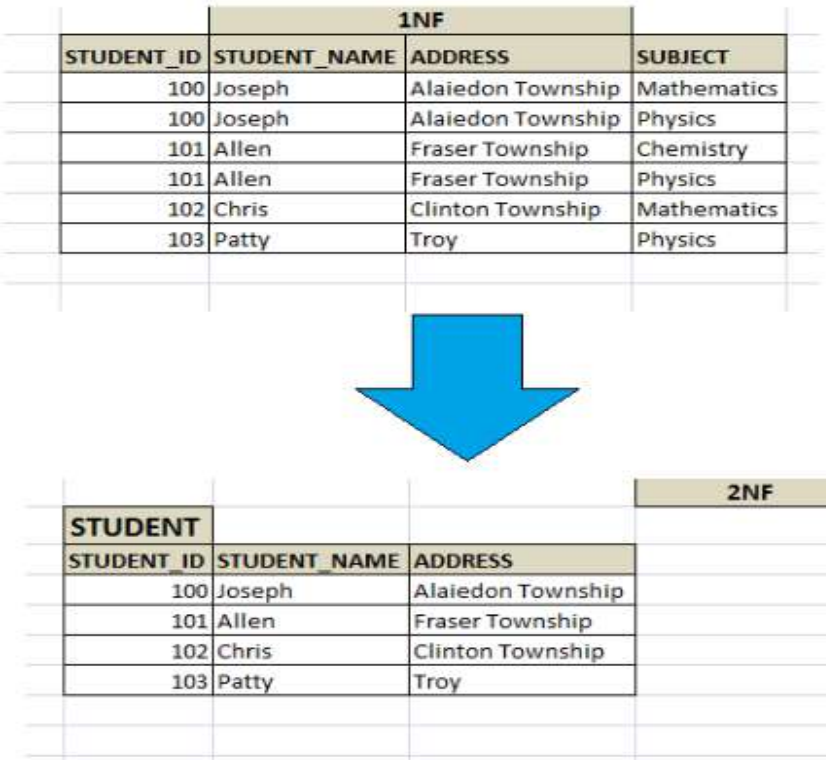
So what we can do here is, apply the third condition of 2NF. Remove Subject from the STUDENT table and create a separate table for it. So the two tables are - STUDENT and SUBJECT. Now the STUDENT table will have only STUDENT information - STUDENT_ID, STUDENT_NAME and ADDRESS. New SUBJECT table will have STUDENT_ID and SUBJECT_NAME.



Now there is no repeating group of columns in STUDENT table and STUDENT_ID is the primary key of STUDENT table. It uniquely identifies the Student name and address which are non key attributes of this table. Hence it satisfies both 1NF and 2NF.

In the new SUBJECT table, Subject names opted by each student. Since same student cannot opt for same subject multiple times in academic year, there will not be any duplicity of data. But Student_ID alone is not unique; hence it cannot be a primary key. Both Student_ID and Subject_Name is unique in this table. Hence both of them together become a primary key. Hence SUBJECT table satisfies 1NF.

There is no non-key attributes in SUBJECT table. Hence we cannot verify second condition of 2NF. According to the third condition of 2NF, we have removed the data which is forming multiple rows and put those details in new table. It also states that there should be relationship between the original table and new table by using foreign key constraint. In the SUBJECT table, STUDENT_ID is derived from STUDENT table. In this table, STUDENT _ID is part of primary key as well as it is a foreign key. Hence we can easily relate both STUDENT and SUBJECT table by using STUDENT_ID. Hence satisfies the third condition of 2NF.



1NF

STUDENT_ID	STUDENT_NAME	ADDRESS	SUBJECT
100	Joseph	Alaiedon Township	Mathematics
100	Joseph	Alaiedon Township	Physics
101	Allen	Fraser Township	Chemistry
101	Allen	Fraser Township	Physics
102	Chris	Clinton Township	Mathematics
103	Patty	Troy	Physics

2NF

STUDENT

STUDENT_ID	STUDENT_NAME	ADDRESS
100	Joseph	Alaiedon Township
101	Allen	Fraser Township
102	Chris	Clinton Township
103	Patty	Troy

SUBJECT

STUDENT_ID	SUBJECT_NAME
100	Mathematics
100	Physics
101	Chemistry
101	Physics
102	Mathematics
103	Physics

If we want to know which all subjects Joseph has opted for, we would query as below:

```
SELECT std.STUDENT_ID,
std.STUDENT_NAME,
sb.SUBJECT
FROM STUDENT std, SUBJECT sb
WHERE std.STUDENT_ID = sb.STUDENT_ID
AND std.STUDENT_NAME = 'Joseph';
```

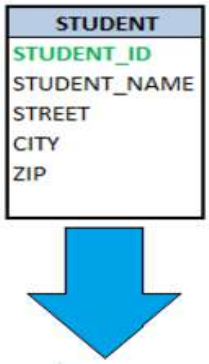
Third Normal Form (3NF)

For a relation to be in third normal form:

- it should meet all the requirements of both 1NF and 2NF

- If there is any columns which are not related to primary key, then remove them and put it in a separate table, relate both the table by means of foreign key i.e.; there should not be any transitive dependency.

Let's add three more columns - STREET, CITY and ZIP to STUDENT table to explain 3NF. Below is the table satisfies conditions for 2NF - there is primary key, no repeating columns, no duplicate datas.



2NF

STUDENT_ID	STUDENT_NAME	STREET	CITY	ZIP
100	Joseph	Alaiedon Village Drive	Alaiedon Township	48035
101	Allen	Fraser Village Drive	Fraser Township	48036
102	Chris	Lakeside Village Drive	Clinton Township	48038
103	Patty	Troy Village Drive	Troy	48039

Here, STREET and CITY have no relation with Student. It's not directly related to student. They fully depend upon zip code. Since Student stays in that area, through zip code, street and city are related to him. This kind of relationship is called transitive dependency. Since its second level of dependency, it is not necessary to store these details in STUDENT table.

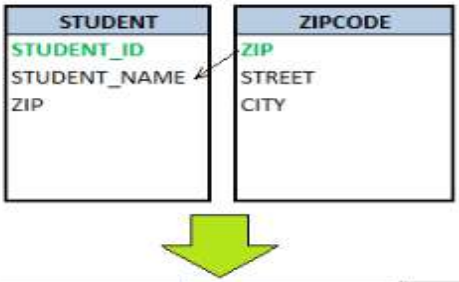
Similarly, if there are multiple students staying in same area, STUDENT table is having huge amount of records and there is a change requested for street or city name, then whole STUDENT table needs to be searched and updated. Imagine, we have to update 'Fraser Village Drive' to Fraser Village Dr'. The Update statement would be

```
UPDATE STUDENT std
SET std.STREET = 'Fraser Village Dr'
WHERE std .STREET =  'Fraser Village Drive';
```

Above query will search whole student table for 'Fraser Village Drive' and then update it to 'Fraser Village Dr'. But searching a huge table and updating the single or multiple records will be a very time consuming, hence affecting the performance of the database.

Instead, if we have these details in a separate table ZIPCODE and is related to STUDENT table using zip? However ZIPCODE table will have comparatively less amount of records and we just have to update ZIPCODE table once. It will automatically reflect in the STUDENT table! Hence making the database and query simpler! And table is in 3NF.

```
UPDATE ZIPCODE z
SET z.STREET = 'Fraser Village Dr'
WHERE z .STREET =  'Fraser Village Drive';
```



3NF

STUDENT

STUDENT_ID	STUDENT_NAME	ZIP
100	Joseph	48035
101	Allen	48036
102	Chris	48038
103	Patty	48039

ZIPCODE

ZIP	STREET	CITY
48035	Alaiedon Village Drive	Alaiedon Township
48036	Fraser Village Drive	Fraser Township
48038	Lakeside Village Drive	Clinton Township
48039	Troy Village Drive	Troy

Now if we have to select the whole address of a student, Chris, we join both STUDENT and ZIPCODE table using ZIP and get the whole address.

```
SELECT std.STUDENT_NAME,
        z.STREET,
        z.CITY,
        z.ZIP
FROM STUDENT std, ZIPCODE z
WHERE std.ZIP = z.ZIP
AND std.STUDENT_NAME = 'Chris';
```

Boyce-Codd Normal Form (3.5NF)

This normal form is also referred as 3.5 normal forms. This normal form

- Meets all the requirement of 3NF
- Any table is said to be in BCNF, if its candidate keys do not have any partial dependency on the other attributes. i.e.; in any table with (x, y, z) columns, if (x, y)->z and z->x then it's a violation of BCNF. If (x, y) are composite keys and (x, y)->z, then there should not be any reverse dependency, directly or partially.

In the above 3NF example, STUDENT_ID is the Primary key in STUDENT table and ZIP is the primary key in the ZIPCODE table. There is no other key column in each of the tables which determines the functional dependency. Hence it's in BCNF form. That is, with STUDENT_ID, we can retrieve STUDENT_NAME and

ZIP from STUDENT table. Similarly, with ZIP value, we can retrieve STREET and CITY from ZIPCODE table

Let us consider another example - consider each student who has taken major subjects has different advisory lecturers. Each student will have different advisory lecturers for same Subjects. There exists following relationship, which is violation of BCNF.

(STUDENT_ID, MAJOR_SUBJECT) -> ADVISORY_LECTURER

ADVISORY_LECTURER -> MAJOR_SUBJECT

i.e. Major_Subject which is a part of composite candidate key is determined non-key attribute of the same table, which is against the rule.

Below table will have all the anomalies too. If we delete any student from below table, it deletes lecturer's information too. If we add any new lecturer/student to the database, it needs other related information also. Also, if we update subject for any student, his lecturer info also needs to be changed, else it will lead to inconsistency.

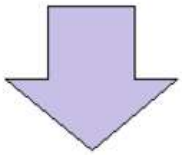
LECTURER
STUDENT_ID
MAJOR_SUBJECT
ADVISORY_LECTURER

STUDENT_ID	MAJOR_SUBJECT	ADVISORY_LECTURER
101	Mathematics	Rose
111	Physics	Joseph
112	Mathematics	Alex
113	Chemistry	Smith
114	Physics	Bosco

Hence we need to decompose the table so that eliminates so that it eliminates such relationship. Now in the new tables below, there are no inter-dependent composite keys (moreover, there is no composite key in both the tables). If we need to add/update/delete any lecturer, we can directly insert record into STUDENT_ADVISOR table, without affecting STUDENT_MAJOR table. If we need to insert/update/delete any subject for a student, then we can directly do it on STUDENT_MAJOR table, without affecting STUDENT_ADVISOR table. When we have both advisor as well as major subject information, then we can directly add/update both the tables. Hence we have eliminated all the anamolies in the database.

STUDENT_ADVISOR
STUDENT_ID
ADVISORY_LECTURER

STUDENT_MAJOR
STUDENT_ID
MAJOR_SUBJECT



STUDENT_ADVISOR	
STUDENT_ID	ADVISORY_LECTURER
101	Rose
111	Joseph
112	Alex
113	Smith
114	Bosco

STUDENT_MAJOR	
STUDENT_ID	MAJOR_SUBJECT
101	Mathematics
111	Physics
112	Mathematics
113	Chemistry
114	Physics

1. Describe about Normalization using functional dependency on (April 2013),(NOV 2015)(April17)

1. Another desirable property in database design is **dependency preservation**.
    - We would like to check easily that updates to the database do not result in illegal relations being created.
    - It would be nice if our design allowed us to check updates without having to compute natural joins.
    - To know whether joins must be computed, we need to determine what functional dependencies may be tested by checking each relation individually.
    - Let *F* be a set of functional dependencies on schema *R*.
    - Let $\{R_1, R_2, \ldots, R_n\}$ be a decomposition of *R*.
    - The **restriction** of *F* to R_i is the set of all functional dependencies in F^+ that include only attributes of R_i.
    - Functional dependencies in a restriction can be tested in one relation, as they involve attributes in one relation schema.
    - The set of restrictions $F_1, F_2, \ldots, F_n$ is the set of dependencies that can be checked efficiently.
    - We need to know whether testing only the restrictions is sufficient.
    - Let $F' = F_1, F_2, \ldots, F_n$.
    - *F'* is a set of functional dependencies on schema *R*, but in general, $F' \neq F$.
    - However, it may be that $F'^+ = F^+$.
    - If this is so, then every functional dependency in *F* is implied by *F'*, and if *F'* is satisfied, then *F* must also be satisfied.
    - A decomposition having the property that $F'^+ = F^+$ is a **dependency-preserving** decomposition.

Boyce-Codd Normal Form

1. A relation schema *R* is in **Boyce-Codd Normal Form (BCNF)** with respect to a set *F* of functional dependencies if for all functional dependencies in F^+ of the form $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds:
    - $\alpha \rightarrow \beta$ is a trivial functional dependency (i.e. $\beta \subseteq \alpha$).

- α is a superkey for schema R.

2. A database design is in BCNF if each member of the set of relation schemas is in BCNF.
3. Let's assess our example banking design:

Customer-schema = (cname, street, ccity)

$cname \rightarrow street\ ccity$

Branch-schema = (bname, assets, bcity)

$bname \rightarrow assets\ bcity$

Loan-info-schema = (bname, cname, loan#, amount)

$loan\# \rightarrow amount\ bname$

Customer-schema and *Branch-schema* are in BCNF.

4. Let's look at *Loan-info-schema*:
   - We have the non-trivial functional dependency $loan\# \rightarrow amount$, and
   - *loan#* is not a superkey.
   - Thus *Loan-info-schema* is not in BCNF.
   - We also have the repetition of information problem.
   - For each customer associated with a loan, we must repeat the branch name and amount of the loan.
   - We can eliminate this redundancy by decomposing into schemas that are all in BCNF.
5. If we decompose into

Loan-schema = (bname, loan#, amount)

Borrow-schema = (cname, loan#)

we have a lossless-join decomposition. (Remember why?)

To see whether these schemas are in BCNF, we need to know what functional dependencies apply to them.

   - For *Loan-schema*, we have $loan\# \rightarrow amount\ bname$ applying.
   - Only trivial functional dependencies apply to *Borrow-schema*.
   - Thus both schemas are in BCNF.

We also no longer have the repetition of information problem. Branch name and loan amount information are not repeated for each customer in this design.

6. Now we can give a general method to generate a collection of BCNF schemas.

result := $\{R\}$;

done := false;

compute F^+;

while (not *done*) **do**

if (there is a schema R_i in *result* that is not in BCNF)

then begin

let $\alpha \rightarrow \beta$ be a nontrivial functional dependency that holds on R_i such that $\alpha \rightarrow R_i$ is not in F^+, and $\alpha \cap \beta = \emptyset$;

result = (*result* $- R_i) \cup (R_i - \beta) \cup (\alpha, \beta)$;

end

else *done* = true;

7. This algorithm generates a lossless-join BCNF decomposition. Why?
   - We replace a schema R_i with $(R_i - \beta)$ and (α, β).
   - The dependency $\alpha \rightarrow \beta$ holds on R_i.
   - $(R_i - \beta) \cap (\alpha, \beta) = \alpha$.
   - So we have $R_1 \cap R_2 \rightarrow R_2$, and thus a lossless join.
8. Let's apply this algorithm to our earlier example of poor database design:

Lending-schema = (bname, assets, bcity, loan#, cname, amount)

The set of functional dependencies we require to hold on this schema are

bname $\rightarrow$ *assets bcity*

loan# $\rightarrow$ *amount bname*

A candidate key for this schema is {*loan#, cname*}.

We will now proceed to decompose:

- The functional dependency
- *bname* $\rightarrow$ *assets bcity*

  holds on *Lending-schema*, but *bname* is not a superkey.

  We replace *Lending-schema* with

  Branch-schema = (bname, assets, bcity)

  Loan-info-schema = (bname, loan#, cname, amount)
- *Branch-schema* is now in BCNF.
- The functional dependency *loan#* $\rightarrow$ *amount bname* olds on *Loan-info-schema*, but *loan#* is not a superkey.

We replace *Loan-info-schema* with

Loan-schema = (bname, loan#, amount)

Borrow-schema = (cname, loan#)

- These are both now in BCNF.
- We saw earlier that this decomposition is both lossless-join and dependency-preserving.

9. Not every decomposition is dependency-preserving.
   - Consider the relation schema
   - *Banker-schema = (bname, cname, banker-name)*
   - The set *F* of functional dependencies is
   - *banker-name* $\rightarrow$ *bname*
   -
   - *cname bname* $\rightarrow$ *banker-name*
   - The schema is not in BCNF as *banker-name* is not a superkey.
   - If we apply our algorithm, we may obtain the decomposition
   - *Banker-branch-schema = (bname, banker-name)*
   - *Cust-banker-schema = (cname, banker-name)*
   - The decomposed schemas preserve only the first (and trivial) functional dependencies.
   - The closure of this dependency does not include the second one.
   - Thus a violation of *cname bname* $\rightarrow$*banker-name* cannot be detected unless a join is computed.

This shows us that not every BCNF decomposition is dependency-preserving.

10. It is not always possible to satisfy all three design goals:
    - BCNF.
    - Lossless join.
    - Dependency preservation.
11. We can see that any BCNF decomposition of *Banker-schema* must fail to preserve
12. *cname bname* $\rightarrow$ *banker-name*
13. **Some Things To Note About BCNF**
    - There is sometimes more than one BCNF decomposition of a given schema.
    - The algorithm given produces only one of these possible decompositions.
    - Some of the BCNF decompositions may also yield dependency preservation, while others may not.
    - Changing the order in which the functional dependencies are considered by the algorithm may change the decomposition.

- For example, try running the BCNF algorithm on
- $R = (A, B, C, D)$

  $A \rightarrow B, C$

  $B \rightarrow D$

  $D \rightarrow B$

Then change the order of the last two functional dependencies and run the algorithm again. Check the two decompositions for dependency preservation.

Third Normal Form

1. When we cannot meet all three design criteria, we abandon BCNF and accept a weaker form called **third normal form (3NF)**.
2. It is always possible to find a dependency-preserving lossless-join decomposition that is in 3NF.
3. A relation schema *R* is in **3NF** with respect to a set *F* of functional dependencies if for all functional dependencies in F^+ of the form $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds:
   a. $\alpha \rightarrow \beta$ is a trivial functional dependency.
   b. α is a superkey for schema *R*.
   c. Each attribute *A* in $\beta - \alpha$ is contained in a candidate key for *R*.
4. A database design is in 3NF if each member of the set of relation schemas is in 3NF.
5. We now allow functional dependencies satisfying only the third condition. These dependencies are called **transitive dependencies**, and are not allowed in BCNF.
6. As all relation schemas in BCNF satisfy the first two conditions only, a schema in BCNF is also in 3NF.
7. BCNF is a more restrictive constraint than 3NF.
8. Our *Banker-schema* decomposition did not have a dependency-preserving lossless-join decomposition into BCNF. The schema was already in 3NF though (check it out).
9. We now present an algorithm for finding a dependency-preserving lossless-join decomposition into 3NF.
10. Note that we require the set *F* of functional dependencies to be in **canonical form**.

let F_c be a **canonical cover** for *F*;

i := 0;

for each functional dependency $\alpha \rightarrow \beta \in F_c$ **do**

if none of the schemas R_j, $1 \leq j \leq i$ contains $\alpha\beta$

then begin

$i := i + 1;$

$R_i := \alpha\beta$

end

if none of the schemas $R_j, 1 \leq j \leq i$ contains a candidate key for R

then begin

$i := i + 1;$

R_i := any candidate key for R

end

return $(R_1, R_2, \ldots, R_i)$

Each relation schema is in . Why? (A proof is given is [Ullman 1988].)

Comparison of BCNF and 3NF

1. We have seen BCNF and 3NF.
   - It is always possible to obtain a 3NF design without sacrificing lossless-join or dependency-preservation.
   - If we do not eliminate all transitive dependencies, we may need to use null values to represent some of the meaningful relationships.
   - Repetition of information occurs.
2. These problems can be illustrated with *Banker-schema*.
   - As *banker-name* $\rightarrow$ *bname* , we may want to express relationships between a banker and his or her branch.

cname	banker-name	bname
Bill	John	SFU
Tom	John	SFU
Mary	John	SFU
null	Tim	Austin

Figure 7.4: An instance of *Banker-schema*.

   - Figure 7.4 shows how we must either have a corresponding value for customer name, or include a null.
   - Repetition of information also occurs.
   - Every occurrence of the banker's name must be accompanied by the branch name.
3. If we must choose between BCNF and dependency preservation, it is generally better to opt for 3NF.

- If we cannot check for dependency preservation efficiently, we either pay a high price in system performance or risk the integrity of the data.
- The limited amount of redundancy in 3NF is then a lesser evil.

4. To summarize, our goal for a relational database design is
   - BCNF.
   - Lossless-join.
   - Dependency-preservation.
5. If we cannot achieve this, we accept
   - 3NF
   - Lossless-join.
   - Dependency-preservation.
6. **A final point:** there is a price to pay for decomposition. When we decompose a relation, we have to use natural joins or Cartesian products to put the pieces back together. This takes computational time.

Fourth Normal Form (4NF)

1. We saw that *BC-schema* was in BCNF, but still was not an ideal design as it suffered from repetition of information. We had the multivalued dependency *cname* $\twoheadrightarrow$ *street ccity*, but no non-trivial functional dependencies.
2. We can use the given multivalued dependencies to improve the database design by decomposing it into **fourth normal form**.
3. A relation schema *R* is in 4NF with respect to a set *D* of functional and multivalued dependencies if for all multivalued dependencies in D^+ of the form $\alpha \twoheadrightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following hold:
   - $\alpha \twoheadrightarrow \beta$ is a trivial multivalued dependency.
   - α is a superkey for schema *R*.
4. A database design is in 4NF if each member of the set of relation schemas is in 4NF.
5. The definition of 4NF differs from the BCNF definition only in the use of multivalued dependencies.
   - Every 4NF schema is also in BCNF.
   - To see why, note that if a schema is not in BCNF, there is a non-trivial functional dependency $\alpha \rightarrow \beta$ holding on *R*, where α is not a superkey.
   - Since $\alpha \rightarrow \beta$ implies $\alpha \twoheadrightarrow \beta$, by the replication rule, *R* cannot be in 4NF.
6. We have an algorithm similar to the BCNF algorithm for decomposing a schema into 4NF:

result := $\{R\}$;

done := false;

compute D^+;

while (not *done*) **do**

if (there is a schema R_i in *result*

that is not in 4NF)

then begin

let $\alpha \twoheadrightarrow \beta$ be a nontrivial multivalued

dependency that holds on R_i such that $\alpha \rightarrow R_i$ is not in D^+, and

$\alpha \cap \beta = \emptyset$;

result = $(result - R_i) \cup (R_i - \beta) \cup (\alpha, \beta)$;

end

else *done* = true;

7. If we apply this algorithm to *BC-schema*:
   - *cname* $\twoheadrightarrow$ *loan#* is a nontrivial multivalued dependency and *cname* is not a superkey for the schema.
   - We then replace *BC-schema* by two schemas:
   - *Cust-loan-schema=(cname, loan#)*
   - *Customer-schema=(cname, street, ccity)*
   - These two schemas are in 4NF.
8. We can show that our algorithm generates only lossless-join decompositions.

- Let *R* be a relation schema and *D* a set of functional and multivalued dependencies on *R*.
- Let R_1 and R_2 form a decomposition of *R*.
- This decomposition is lossless-join if and only if at least one of the following multivalued dependencies is in D^+:
- $R_1 \cap R_2 \twoheadrightarrow R_1$
- $R_1 \cap R_2 \twoheadrightarrow R_2$
- We saw similar criteria for functional dependencies.
- This says that for **every** lossless-join decomposition of *R* into two schemas and , one of the two above dependencies must hold.
- You can see, by inspecting the algorithm, that this must be the case for every decomposition.

9. Dependency preservation is not as simple to determine as with functional dependencies.

- Let *R* be a relation schema.
- Let $R_1, R_2, \ldots R_n$ be a decomposition of *R*.
- Let *D* be the set of functional and multivalued dependencies holding on *R*.
- The **restriction** of *D* to R_i is the set D_i consisting of:
    - All functional dependencies in D^+ that include only attributes of R_i.
    - All multivalued dependencies of the form $\alpha \twoheadrightarrow \beta \cap R_i$ where $\alpha \subseteq R_i$ and $\alpha \twoheadrightarrow \beta$ is in D^+.
- A decomposition of schema *R* is dependency preserving with respect to a set *D* of functional and multivalued dependencies if for every set of relations $r_1(R_1), r_2(R_2), \ldots r_n(R_n)$ such that for all *i*, r_i satisfies D_i, there exists a relation *r*(*R*) that satisfies *D* and for which $r_i = \Pi_{R_i}(r)$ for all *i*.

10. What does this formal statement say? It says that a decomposition is dependency preserving if for every set of relations on the decomposition schema satisfying only the restrictions on *D* there exists a relation *r* on the entire schema *R* that the decomposed schemas can be derived from, and that *r* also satisfies the functional and multivalued dependencies.
11. We'll do an example using our decomposition algorithm and check the result for dependency preservation.
    - Let *R*=(*A*,*B*,*C*,*G*,*H*,*I*).
    - Let D be
    - $A \twoheadrightarrow B$
    - $B \twoheadrightarrow HI$
    - $CG \rightarrow H$
    - *R* is not in 4NF, as we have $A \twoheadrightarrow B$ and *A* is not a superkey.
    - The algorithm causes us to decompose using this dependency into
    - $R_1 = (A, B)$
    - $R_2 = (A, C, G, H, I)$
    - R_1 is now in 4NF, but R_2 is not.
    - Applying the multivalued dependency $CG \twoheadrightarrow H$ (how did we get this?), our algorithm then decomposes R_2 into
    - $R_3 = (C, G, H)$
    - $R_4 = (A, C, G, I)$
    - R_3 is now in 4NF, but R_4 is not.

- Why? As $A \twoheadrightarrow HI$ is in D^+ (why?) then the restriction of this dependency to R_4 gives us $A \twoheadrightarrow I$.
- Applying this dependency in our algorithm finally decomposes R_4 into
- $$R_5 = (A, I)$$
- $$R_6 = (A, C, G)$$
- The algorithm terminates, and our decomposition is R_1, R_3, R_5 and R_6.

12. Let's analyze the result.

r_1:

A	B
a_1	b_1
a_2	b_1

r_2:

C	G	H
c_1	g_1	h_1
c_2	g_2	h_2

r_3:

A	I
a_1	i_1
a_1	i_2

r_4:

A	C	G
a_1	c_1	g_1
a_2	c_2	g_2

Figure 7.9: Projection of relation *r* onto a 4NF decomposition of *R*.

- This decomposition is not dependency preserving as it fails to preserve $B \twoheadrightarrow HI$.
- Figure 7.9 (textbook 6.14) shows four relations that may result from projecting a relation onto the four schemas of our decomposition.
- The restriction of *D* to (*A*,*B*) is $A \twoheadrightarrow B$ and some trivial dependencies.
- We can see that r_1 satisfies $A \twoheadrightarrow B$ as there are no pairs with the same *A* value.
- Also, r_2 satisfies all functional and multivalued dependencies since no two tuples have the same value on any attribute.
- We can say the same for r_3 and r_4.
- So our decomposed version satisfies all the dependencies in the restriction of *D*.
- However, there is no relation *r* on (*A*,*B*,*C*,*G*,*H*,*I*) that satisfies *D* and decomposes into r_1, r_2, r_3 and r_4.
- Figure 7.10 (textbook 6.15) shows $r = r_1 \bowtie r_2 \bowtie r_3 \bowtie r_4$.
- Relation *r* does not satisfy $B \twoheadrightarrow HI$.
- Any relation *s* containing *r* and satisfying $B \twoheadrightarrow HI$ must include the tuple $(a_2, b_1, c_2, g_2, h_1, i_1)$.
- However, $\Pi_{CGH}(s)$ includes a tuple (c_2, g_2, h_1) that is not in r_2.
- Thus our decomposition fails to detect a violation of $B \twoheadrightarrow HI$.

A	B	C	G	H	I
a_1	b_1	c_1	g_1	h_1	i_1
a_2	b_1	c_2	g_2	h_2	i_2

Figure 7.10: A relation *r*(*R*) that does not satisfy $B \twoheadrightarrow HI$.

13. We have seen that if we are given a set of functional and multivalued dependencies, it is best to find a database design that meets the three criteria:
    - 4NF.
    - Dependency Preservation.
    - Lossless-join.
14. If we only have functional dependencies, the first criteria is just BCNF.
15. We cannot always meet all three criteria. When this occurs, we compromise on 4NF, and accept BCNF, or even 3NF if necessary, to ensure dependency preservation.

7. Explain referential integrity in detail (Nov 2010)

Referential integrity

An example of a database that has not enforced **referential integrity**. In this example, there is a foreign key (**artist_id**) value in the album table that references a non-existent artist — in other words there is a foreign key value with no corresponding primary key value in the referenced table. What happened here was that there was an artist called "Aerosmith", with an **artist_id** of "4", which was deleted from the artist table. However, the album "Eat the Rich" referred to this artist. With referential integrity enforced, this would not have been possible.

Referential integrity in a relational database is consistency between coupled tables. Referential integrity is usually enforced by the combination of a primary key or candidate key (alternate key) and a foreign key. For referential integrity to hold, any field in a table that is declared a foreign key can contain only values from a parent table's primary key or a candidate key. For instance, deleting a record that contains a value referred to by a foreign key in another table would break referential integrity. The relational database management system (RDBMS) enforces referential integrity, normally either by deleting the foreign key rows as well to maintain integrity, or by returning an error and not performing the delete. Which method is used would be defined by the definition of the referential integrity constraint.

Example

An employee database stores the department in which each employee works. The field "Department Number" in the Employee table is declared a foreign key, and it refers to the field "Index" in the Department table which is declared a primary key. Referential integrity would be broken by deleting a department from the Department table if employees listed in the Employee table are listed as working for that department, unless those employees are moved to a different department at the same time.

Database trigger

A **database trigger** is procedural code that is automatically executed in response to certain events on a particular table in a database. Triggers can restrict access to specific data, perform logging, or audit data modifications.

There are two classes of triggers, they are either "row triggers" or "statement triggers". With row triggers you can define an action for every row of a table, while statement triggers occur only once per INSERT, UPDATE, or DELETE statement. Triggers cannot be used to audit data retrieval via SELECT statements.

Each class can be of several types. There are "BEFORE triggers" and "AFTER triggers" which identifies the time of execution of the trigger. There is also an "INSTEAD OF trigger" which is a trigger that will execute instead of the triggering statement.

There are typically three triggering events that cause triggers to 'fire':

- INSERT event (as a new record is being inserted into the database).
- UPDATE event (as a record is being changed).
- DELETE event (as a record is being deleted).

The trigger is used to automate DML condition process.

The major features and effects of database triggers are that they:

- do not accept parameters or arguments (but may store affected-data in temporary tables)
- cannot perform commit or rollback operations because they are part of the triggering SQL statement (only through autonomous transactions)
- can cause mutating table errors, if they are poorly written.

Triggers in Oracle

In addition to triggers that fire when data is modified, Oracle 9i supports triggers that fire when schema objects (that is, tables) are modified and when user logon or logoff events occur. These trigger types are referred to as "Schema-level triggers".

Schema-level triggers

- After Creation
- Before Alter
- After Alter
- Before Drop
- After Drop
- Before Logoff
- After Logon

Triggers in Microsoft SQL Server

Microsoft SQL Server supports triggers either after or instead of an insert, update, or delete operation.

'***Microsoft SQL Server*** *supports triggers on tables and views with the constraint that a view can be referenced only by an INSTEAD OF trigger.*

Microsoft SQL Server 2005 introduced support for Data Definition Language (DDL) triggers, which can fire in reaction to a very wide range of events, including:

- Drop table
- Create table
- Alter table
- Login events

Performing conditional actions in triggers (or testing data following modification) is done through accessing the temporary *Inserted* and *Deleted* tables.

Triggers in PostgreSQL

PostgreSQL introduced support for triggers in 1997. The following functionality in SQL:2003 is not implemented in PostgreSQL:

- SQL allows triggers to fire on updates to specific columns; PostgreSQL does not support this feature.
- The standard allows the execution of a number of other SQL statements than SELECT, INSERT, UPDATE, such as CREATE TABLE as the triggered action.

Triggers in MySQL

MySQL 5.0 introduced support for triggers. Some of the triggers MYSQL supports are

- INSERT Trigger
- UPDATE Trigger
- DELETE Trigger

The SQL:2003 standard mandates that triggers give programmers access to record variables by means of a syntax such as *REFERENCING NEW AS n*. For example, if a trigger is monitoring for changes to a salary column one could write a trigger like the following:

```
CREATE TRIGGER salary_trigger
  BEFORE UPDATE ON employee_table
  REFERENCING NEW ROW AS n, OLD ROW AS o
  FOR EACH ROW
  IF n.salary <> o.salary THEN
    --make desired changes;
  END IF;
```

8. Explain any one Database system of your own with normalization (Nov 2010)

Normalization is a method for organizing data elements in a database into tables.

Normalization Avoids

- Duplication of Data – The same data is listed in multiple lines of the database
- Insert Anomaly – A record about an entity cannot be inserted into the table without first inserting information about another entity – Cannot enter a customer without a sales order
- Delete Anomaly – A record cannot be deleted without deleting a record about a related entity. Cannot delete a sales order without deleting all of the customer's information.
- Update Anomaly – Cannot update information without changing information in many places. To update customer information, it must be updated for each sales order the customer has placed

Normalization is a three stage process – After the first stage, the data is said to be in first normal form, after the second, it is in second normal form, after the third, it is in third normal form

Before Normalization

1. Begin with a list of all of the fields that must appear in the database. Think of this as one big table.
2. Do not include computed fields
3. One place to begin getting this information is from a printed document used by the system.
4. Additional attributes besides those for the entities described on the document can be added to the database.

Before Normalization – Example

See Sales Order from below:

Sales Order

Fiction Company
202 N. Main
Mahattan, KS 66502

CustomerNumber:	*1001*	**Sales Order Number:**	405
Customer Name:	*ABC Company*	**Sales Order Date:**	2/1/2000
Customer Address:	*100 Points*	**Clerk Number:**	*210*
	Manhattan, KS 66502	**Clerk Name:**	*Martin Lawrence*

Item Ordered	Description	Quantity	Unit Price	Total
800	*widgit small*	*40*	*60.00*	*2,400.00*
801	*tingimajigger*	*20*	*20.00*	*400.00*
805	*thingibob*	*10*	*100.00*	*1,000.00*
	Order Total			*3,800.00*

Fields in the original data table will be as follows:

SalesOrderNo, Date, CustomerNo, CustomerName, CustomerAdd, ClerkNo, ClerkName, ItemNo, Description, Qty, UnitPrice

Think of this as the baseline – one large table

Normalization: First Normal Form

- Separate Repeating Groups into New Tables.
- ***Repeating Groups*** Fields that may be repeated several times for one document/entity
- Create a new table containing the repeating data
- The primary key of the new table (repeating group) is always a composite key; Usually document number and a field uniquely describing the repeating line, like an item number.

First Normal Form Example

The new table is as follows:

SalesOrderNo, ItemNo, Description, Qty, UnitPrice

The repeating fields will be removed from the original data table, leaving the following.

SalesOrderNo, Date, CustomerNo, CustomerName, CustomerAdd, ClerkNo, ClerkName

These two tables are a database in first normal form

What if we did not Normalize the Database to First Normal Form?

Repetition of Data – SO Header data repeated for every line in sales order.

Normalization: Second Normal Form

- Remove Partial Dependencies.
- ***Functional Dependency*** The value of one attribute in a table is determined entirely by the value of another.
- ***Partial Dependency*** A type of functional dependency where an attribute is functionally dependent on only part of the primary key (primary key must be a composite key).
- Create separate table with the functionally dependent data and the part of the key on which it depends. Tables created at this step will usually contain descriptions of resources.

Second Normal Form Example

The new table will contain the following fields:

ItemNo, Description

All of these fields except the primary key will be removed from the original table. The primary key will be left in the original table to allow linking of data:

SalesOrderNo, ItemNo, Qty, UnitPrice

Never treat price as dependent on item. Price may be different for different sales orders (discounts, special customers, etc.)

Along with the unchanged table below, these tables make up a database in second normal form:

SalesOrderNo, Date, CustomerNo, CustomerName, CustomerAdd, ClerkNo, ClerkName

What if we did not Normalize the Database to Second Normal Form?

- Repetition of Data – Description would appear every time we had an order for the item
- Delete Anomalies – All information about inventory items is stored in the SalesOrderDetail table. Delete a sales order, delete the item.
- Insert Anomalies – To insert an inventory item, must insert sales order.
- Update Anomalies – To change the description, must change it on every SO.

Normalization: Third Normal Form

- Remove transitive dependencies.
- ***Transitive Dependency*** A type of functional dependency where an attribute is functionally dependent on an attribute other than the primary key. Thus its value is only indirectly determined by the primary key.

- Create a separate table containing the attribute and the fields that are functionally dependent on it. Tables created at this step will usually contain descriptions of either resources or agents. Keep a copy of the key attribute in the original file.

Third Normal Form Example

The new tables would be:

CustomerNo, CustomerName, CustomerAdd

ClerkNo, ClerkName

All of these fields except the primary key will be removed from the original table. The primary key will be left in the original table to allow linking of data as follows:

SalesOrderNo, Date, CustomerNo, ClerkNo

Together with the unchanged tables below, these tables make up the database in third normal form.

ItemNo, Description

SalesOrderNo, ItemNo, Qty, UnitPrice

What if we did not Normalize the Database to Third Normal Form?

- Repetition of Data – Detail for Cust/Clerk would appear on every SO
- Delete Anomalies – Delete a sales order, delete the customer/clerk
- Insert Anomalies – To insert a customer/clerk, must insert sales order.
- Update Anomalies – To change the name/address, etc, must change it on every SO.

Completed Tables in Third Normal Form

Customers: CustomerNo, CustomerName, CustomerAdd

Clerks: ClerkNo, ClerkName

Inventory Items: *ItemNo*, Description

Sales Orders: SalesOrderNo, Date, CustomerNo, ClerkNo

SalesOrderDetail: SalesOrderNo, ItemNo, Qty, UnitPrice

11 MARKS

1. Describe about Normalization using functional dependency **(April 2013)(NOV 2015)[April 17][Question No. 06]**
2. Explain referential integrity in detail **(Nov 2010) [Question No. 07]**
3. Explain any one Database system of your own with normalization (**Nov 2010) [Question No. 08]**

4.What are functional dependencies? Explain each of them with a suitable applicative example. [April 2017] (Question Number:06)

5.Define Normalization and explain different types of normalization with suitable examples. **[NOV 2017], [APR 2018] [DEC 2018] (MAY 2019) [Refer Q.No. 6]**

6.List the three design goals for relational databases, and explain why each is desirable. **[APR 2018]**

7.Explain decomposition using multivalued dependency **[NOV 2017]**

8. Explain 4 NF and 5 NF with examples **(11 marks)APRIL 2010)**

9.Discuss Assertions, Triggers, Authentication (11 Marks) **APRIL 2010, [DEC 2016]**

10.Explain any one database system of your own with normalization **(11 marks) NOV 2010)**

11.Explain referential integrity in detail(**11 marks) NOV 2010)**

12. Explain database design process in detail. **(11 marks)APRIL 2011)**

13. Explicate the Algorithms for Decomposition**. [DEC 2016], (MAY 2019)**